

งาน

1. มานี มี arrays 2 arrays ที่จัดเก็บ ตามลำดับ คือ $a_1 [1, \dots, n]$, $a_2 [1, \dots, n + 1]$

โดยใน a_1 ประกอบด้วยตัวเลขบวกที่แตกต่างกัน และ a_2 ประกอบด้วยข้อมูลที่ได้จาก a_1 แต่มีการใส่ 0 ลงใน a_2

ซึ่งมานีต้องการทราบตำแหน่ง (index) ของ 0 แต่ มานี ไม่มีเวลาเพียงพอที่จะทำการค้นหาข้อมูลใน array ดังนั้นเธอต้องการมองหาอัลกอริทึมที่รวดเร็ว ซึ่งนักศึกษาต้องออกแบบ algorithm ที่สามารถวิเคราะห์ข้อมูลได้อย่างรวดเร็ว ในการค้นหาดัชนีของ 0 ใน a_2 ซึ่งเวลาในการทำงาน ต้องไม่มากกว่า $O(\log n)$ มาก

เช่น . $a_1 : 1, 3, 4, 6, 7, 8, 9, 20$

$a_2 : 1, 3, 0, 4, 6, 7, 8, 9, 20.$

ซึ่ง algorithm ต้องคืนตำแหน่ง ที่ 3 ของค่า 0

```
public boolean equals(Stack s1, Stack s2) {  
    // กรณี Stack ทั้งคู่ว่างพร้อมกัน ถือว่าเหมือนกัน  
    if (s1.isEmpty() && s2.isEmpty()) {  
        return true;  
    }  
    // กรณีมี Stack ใด Stack หนึ่งว่างเพียงอันเดียว แสดงว่าขนาดไม่เท่า  
    // กัน  
    if (s1.isEmpty() || s2.isEmpty()) {  
        return false;  
    }  
  
    // ดึงข้อมูลออกมาเปรียบเทียบ  
    Object val1 = s1.pop();  
    Object val2 = s2.pop();  
  
    boolean isSame = false;  
    // ตรวจสอบค่าของตัวปัจจุบัน  
    if (val1.equals(val2)) {  
        // เรียกซ้ำเพื่อเช็คตัวถัดไป  
        isSame = equals(s1, s2);  
    }  
}
```

```
// คีนค่าข้อมูลกลับเข้า Stack เพื่อรักษาโครงสร้างเดิม
s1.push(val1);
s2.push(val2);

return isSame;
}
```

2. จาก function ที่กำหนด ซึ่งเป็น function ที่ทำการแบ่ง array ออกเป็น 2 arrays ดังนั้นให้นักศึกษาเขียนโปรแกรมในการรวม array ทั้ง 2 arrays ที่มีการจัดเรียงแล้ว

```
public int getSize(Queue q) {
    Queue tempQ = new Queue();
    int count = 0;

    // ย้ายข้อมูลออกจาก q เพื่อนับจำนวน
    while (!q.isEmpty()) {
        tempQ.enqueue(q.dequeue());
        count++;
    }
}
```

```

// คีนค่าข้อมูลกลับเข้า q เดิม
while (!tempQ.isEmpty()) {
    q.enqueue(tempQ.dequeue());
}

return count;
}

int middle = values.length/2; // divide array into two arrays of half size
int[] left = new int[middle];
for (int i=0; i<middle; i++) {
    left[i] = values[i];
}
int[] right = new int[values.length-middle];
for (int i=0; i<values.length-middle; i++) {
    right[i] = values[middle+i];
}
sort(left); //recursively call sorting function on each smaller array
sort(right);

```

3. จาก

A[0....7] =

15	3	9	31	11	17	7	23
----	---	---	----	----	----	---	----

3.1 แสดง insertion sort ทุกขั้นตอน

3.2 แสดง Quick sort ทุกขั้นตอน โดย pivot คือ ตำแหน่งสุดท้าย ของ array

```
public void trimFirstHalf() {  
    // ตรวจสอบว่าโหนดมีจำนวนเป็นเลขคู่และไม่ว่างเปล่า  
    if (list_size > 0 && list_size % 2 == 0) {  
        int half = list_size / 2;  
        SNode current = head;  
  
        // เลื่อนตำแหน่งไปตามจำนวนครึ่งแรก  
        for (int i = 0; i < half; i++) {  
            current = current.next;  
        }  
  
        // ตัดครึ่งแรกออก โดยเปลี่ยน head ไปชี้ที่ครึ่งหลัง  
        head = current;  
        list_size = half;  
    }  
}
```

4. แสดงการเพิ่มค่า 5, 28, 19, 15, 20, 33, 12, 17, 10 ลงใน ตารางแฮช hash table ที่แก้ปัญหาการชนกัน โดยการ chaining ซึ่ง

กำหนดให้ตารางมี 9 ช่อง และ แฮชฟังก์ชัน (hash function) $h(k) = k \bmod 9$

```
public int findMin() {  
    if (head == null) {  
        throw new IllegalStateException("List is empty");  
    }  
  
    int minVal = head.key;  
    SNode current = head.next;  
  
    while (current != null) {  
        if (current.key < minVal) {  
            minVal = current.key;  
        }  
        current = current.next;  
    }  
  
    return minVal;  
}
```

5. ให้เขียนโปรแกรม Stack โดยใช้ linked list ในการจัดการข้อมูล

```
public class Node {  
    public Object data;  
    public Node next;  
  
    public Node(Object data) {
```

```

        this.data = data;
        this.next = null;
    }
}

public class LinkedStack {
    private Node top;

    public LinkedStack() {
        this.top = null;
    }

    public boolean isEmpty() {
        return top == null;
    }

    public void push(Object x) {
        Node newNode = new Node(x);
        newNode.next = top;
        top = newNode;
    }

    public Object pop() {
        if (isEmpty()) {
            System.out.println("Stack Underflow");
            return null;
        }
        Object item = top.data;
        top = top.next;
        return item;
    }

    public Object peek() {
        if (isEmpty()) {
            return null;
        }
        return top.data;
    }
}

```